BELLA ECUATOGUINEANA · BIOCOSMÉTICA S.L.U.

Mettre la boutique en ligne sur o2switch

Procédure complète, de la préparation en local jusqu'à la première commande réelle. Laravel 12 sur le sous-domaine `api.`, le site React sur le domaine principal.

---

8 PHASES 2 À 3 HEURES ACCÈS SSH OU TERMINAL CPANEL

Dans tout ce document, remplace **TONDOMAINE.COM** par ton nom de domaine et **TON_USER** par ton identifiant cPanel.
Buena Esperanza 1, R15, Malabo, Guinée Équatoriale — Laravel 12 · React 19 · MariaDB.

Sommaire

00 Prérequis et architecture

01 Préparation en local

02 Configuration cPanel

03 Envoi des fichiers

04 Installation côté serveur

05 Certificat HTTPS

06 Vérification

07 Mises à jour

– Dépannage

– Ce qui reste après la mise en ligne

00 Prérequis et architecture

Trois choses à réunir avant de commencer. Si l'une manque, la suite ne pourra pas aboutir.

- [ ] Le domaine est actif chez o2switch, ou ses DNS y pointent déjà
- [ ] Tu as tes identifiants cPanel
- [ ] Tu connais ton adresse IP publique — relève-la sur monip.org

ARCHITECTURE VISÉE

Le site React est servi depuis `public_html`. Laravel vit dans un dossier **hors du web**, et seul son sous-dossier `public` est exposé, via le sous-domaine `api.`. Le fichier `.env` — qui contient les mots de passe de la base — reste ainsi inaccessible depuis un navigateur.

ARBORESCENCE SUR LE SERVEUR

```
/home/TON_USER/
├── public_html/          → TONDOMAINE.COM — le site React
└── laravel_sandy/        → hors du web
    └── public/           → api.TONDOMAINE.COM — l'API Laravel
```

01 Préparation en local

1.1 — Supprimer les fichiers parasites

L'archive `vendor.zip` pèse 34 Mo et n'a rien à faire sur le serveur.

```powershell
Remove-Item e:\Pjets_dan\Sandy\backend\vendor.zip
```

1.2 — Remettre les compteurs à zéro

Les commandes de test ont consommé des numéros de référence. Sans cette étape, la première commande d'un vrai client ne s'appellerait pas `SB-2026-0001`.

```powershell
cd e:\Pjets_dan\Sandy\backend
php artisan tinker --execute="DB::table('order_items')->delete(); DB::table('payments')->delete();
DB::table('orders')->delete(); DB::statement('ALTER TABLE orders AUTO_INCREMENT = 1'); echo 'ok';"
php artisan db:seed --class=CatalogSeeder
```

1.3 — Générer la clé de production

```powershell
php artisan key:generate --show
```

Copie la ligne `base64:…` dans un bloc-notes, tu la colleras à l'étape 4.3. L'option `--show` affiche la clé sans modifier ton fichier local.

1.4 — Construire le site React

Crée d'abord `frontend\.env.production` avec l'adresse de ton API :

```frontend/.env.production
VITE_API_URL=https://api.TONDOMAINE.COM
```

```powershell
cd e:\Pjets_dan\Sandy\frontend
npm run build
findstr /C:"api.TONDOMAINE.COM" dist\assets\*.js
```

> **LE PIÈGE LE PLUS COÛTEUX**
>
> Vite écrit cette adresse **en dur dans le fichier JavaScript** au moment du build. La commande `findstr` doit trouver quelque chose. Si elle ne renvoie rien, le site en ligne appellera `localhost:8000` et la page restera vide, sans message d'erreur visible.
>
> Corollaire : le jour où tu changes de domaine, il faut **reconstruire**, pas seulement modifier un réglage sur le serveur.

1.5 — Préparer l'archive du backend

Compresse le dossier `backend` en excluant ce que le serveur reconstruira :

À EXCLURE	POURQUOI
`vendor/`	82 Mo — Composer l'installera sur le serveur
`.env`	Contient les réglages locaux ; on en écrit un autre en 4.3
`.git/`	Historique inutile en production
`storage/logs/*`	Journaux de développement

> **CONTRÔLE**
>
> L'archive doit peser **moins de 1 Mo**. Si elle en fait 20, `vendor/` a été inclus — recommencer.

02 Configuration cPanel

2.1 — Version de PHP

MultiPHP Manager → sélectionner **PHP 8.2** ou plus, pour le domaine principal **et** pour `api.`

2.2 — Extensions PHP

Sélecteur PHP → *Extensions*. Vérifier la présence de :

```
EXTENSIONS REQUISES
pdo_mysql   mbstring   openssl   curl
fileinfo    gd         zip       xml      tokenizer
```

> **« PDO_MYSQL SKIPPED AS CONFLICTING »**
>
> Message normal, à ignorer. cPanel propose deux variantes du même pilote — `nd_pdo_mysql` (moderne, activé par défaut) et `pdo_mysql` (ancien) — qui s'excluent mutuellement. Laravel se satisfait de l'une ou l'autre. Même logique pour `nd_mysqli` face à `mysqli`.
>
> `fileinfo` en revanche est indispensable : sans lui, la validation des photos produits échoue au téléversement.

2.3 — Autoriser son IP en SSH

Autorisation SSH → ajouter l'IP relevée en phase 0. Si l'adresse change souvent, on peut ignorer cette étape et tout faire depuis le **Terminal** intégré à cPanel, qui ne demande aucune autorisation.

2.4 — Créer la base de données

MySQL® Databases, dans cet ordre :

1. **Create New Database** → `sandy`
2. **Add New User** → mot de passe long, à noter
3. **Add User To Database** → cocher **ALL PRIVILEGES**

cPanel préfixe les noms. Relever les valeurs **exactes** pour l'étape 4.3 :

```
À NOTER
Base         TON_USER_sandy
Utilisateur  TON_USER_sandy
Mot de passe …
```

2.5 — Créer le sous-domaine de l'API

Domains → **Create A New Domain** :

DOMAINE	`api.TONDOMAINE.COM`
SHARE DOCUMENT ROOT	décoché
DOCUMENT ROOT	`/home/TON_USER/laravel_sandy/public`

> **L'ÉTAPE À NE PAS RATER**
>
> Le *Document Root* pointe sur `/public`, jamais sur la racine de Laravel. C'est ce qui empêche quiconque d'ouvrir `api.TONDOMAINE.COM/.env` et de lire les mots de passe de la base de données.

03 Envoi des fichiers

3.1 — Le backend

Gestionnaire de fichiers → `/home/TON_USER/` → **Upload** le `.zip` → clic droit → **Extract**.

Renommer le dossier obtenu en `laravel_sandy`. Ce chemin doit exister :

> CHEMIN DE CONTRÔLE
>
> `/home/TON_USER/laravel_sandy/public/index.php`

3.2 — Le site React

Gestionnaire de fichiers → `public_html/` → le vider, puis envoyer **le contenu** de `frontend\dist\` — c'est-à-dire `index.html` et le dossier `assets/`, et non le dossier `dist` lui-même.

> **POURQUOI UNE ARCHIVE PLUTÔT QUE DU FTP**
>
> Le backend représente quelques centaines de fichiers, `vendor/` en compterait près de quinze mille. Chaque fichier envoyé en FTP demande un aller-retour réseau : un seul `.zip` décompressé sur place prend quelques minutes là où le transfert fichier par fichier prendrait des heures, avec un risque de coupure en cours de route.

04 Installation côté serveur

Ouvrir le **Terminal** dans cPanel, ou se connecter en SSH.

4.1 — Vérifier la version de PHP en ligne de commande

```
TERMINAL
php -v
```

> **PIÈGE CLASSIQUE DE CPANEL**
>
> La version de PHP utilisée par le **site web** et celle de la **ligne de commande** sont souvent différentes. Si `php -v` affiche 7.x ou 8.0, toutes les commandes qui suivent doivent utiliser le chemin complet : `/opt/cpanel/ea-php82/root/usr/bin/php`

En profiter pour contrôler les extensions réellement chargées — plus fiable que les cases cochées dans l'interface :

```
TERMINAL
php -m | grep -i -E "pdo|mysql|mbstring|fileinfo|gd|curl|zip"
```

4.2 — Installer les dépendances

```
TERMINAL
cd ~/laravel_sandy
composer install --no-dev --optimize-autoloader
```

Le `--no-dev` écarte PHPUnit, Faker, Pint et Sail : inutiles en production, et une quarantaine de mégaoctets économisés.

> **SI « ALLOWED MEMORY SIZE EXHAUSTED »**
>
> Relancer en levant la limite mémoire : `php -d memory_limit=-1 /usr/local/bin/composer install --no-dev --optimize-autoloader`

4.3 — Écrire le fichier .env

```
TERMINAL
nano ~/laravel_sandy/.env
```

LARAVEL_SANDY/.ENV

```
APP_NAME="Bella Ecuatoguineana"
APP_ENV=production
APP_KEY=base64:LA_CLE_DE_L_ETAPE_1.3
APP_DEBUG=false
APP_URL=https://api.TONDOMAINE.COM

APP_LOCALE=es
APP_FALLBACK_LOCALE=es

LOG_CHANNEL=stack
LOG_LEVEL=error

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=TON_USER_sandy
DB_USERNAME=TON_USER_sandy
DB_PASSWORD=ton_mot_de_passe

SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database

FRONTEND_URL=https://TONDOMAINE.COM

PAY_BANK_TRANSFER_ENABLED=true
BANK_NAME=BANGE
BANK_ACCOUNT_NAME="Biocosmética S.L.U."
BANK_ACCOUNT_NUMBER=TON_RIB
BANK_SWIFT=TON_SWIFT

PAY_FEDAPAY_ENABLED=false
FEDAPAY_ENV=sandbox
FEDAPAY_SECRET_KEY=
FEDAPAY_WEBHOOK_SECRET=
```

`Ctrl+O` puis `Entrée` pour enregistrer, `Ctrl+X` pour sortir.

LIGNE CRITIQUE	CE QUI ARRIVE SI ELLE EST FAUSSE
`APP_DEBUG=false`	Avec `true`, la moindre erreur affiche publiquement la trace d'exécution **et le contenu du `.env`**, mots de passe compris. C'est la fuite de données la plus courante sur les projets Laravel.
`APP_URL`	Fabrique les adresses des photos produits. Faux ou oublié : **aucune image ne s'affiche** sur la boutique.
`FRONTEND_URL`	Sert au filtrage CORS. Une différence — un `/` final, un `http` au lieu de `https` — et **toutes** les requêtes du site sont bloquées par le navigateur.

4.4 — Créer les tables et le catalogue

```
TERMINAL
cd ~/laravel_sandy
php artisan migrate --force
php artisan db:seed --class=CatalogSeeder
```

`--force` est obligatoire en production : sans lui, Laravel refuse de toucher à la base sans confirmation interactive.

4.5 — Créer le compte administrateur

```
TERMINAL
php artisan tinker --execute="App\Models\User::create(['name' => 'Admin', 'email' =>
'admin@TONDOMAINE.COM', 'password' => 'UnMotDePasseLongEtUnique']);"
```

> **DEUX RÈGLES**
>
> Choisir un mot de passe **différent de celui du poste local**. Et ne pas l'entourer de `bcrypt()` : le modèle `User` le chiffre déjà tout seul, un double chiffrement rendrait la connexion impossible sans message clair.

4.6 — Les permissions d'écriture

```
TERMINAL
cd ~/laravel_sandy
chmod -R 775 storage bootstrap/cache
mkdir -p public/images/products
chmod -R 775 public/images/products
```

Sans ces droits : erreur 500 dès le premier accès, et le téléversement des photos échoue depuis le back-office.

05 Certificat HTTPS

SSL/TLS Status → cocher le domaine principal **et** `api.TONDOMAINE.COM` → **Run AutoSSL**.

> **LES DEUX, PAS UN SEUL**
>
> Tant que le certificat n'est pas actif sur les deux, un site servi en `https` refusera d'appeler une API en `http` : le navigateur bloque la requête et affiche une page vide, sans explication lisible. C'est aussi une exigence de BANGE pour l'ouverture d'un TPV virtual.

06 Vérification

Dans cet ordre — chaque test suppose que le précédent est passé.

- [ ] `api.TONDOMAINE.COM/api/ping` renvoie `{"message":"pong"}`
- [ ] `api.TONDOMAINE.COM/api/products` renvoie la liste des 20 produits
- [ ] `TONDOMAINE.COM` affiche le site, catalogue rempli
- [ ] La console du navigateur (F12) ne montre aucune erreur rouge
- [ ] `TONDOMAINE.COM/#admin` ouvre la connexion, puis le back-office
- [ ] Un produit ajouté avec photo affiche bien son image
- [ ] Une commande complète produit la référence `SB-2026-0001`

07 Mises à jour

Le travail d'installation ne se refait pas. Les déploiements suivants se limitent à ceci.

Modifier le backend

Envoyer uniquement les fichiers changés — `app/`, `config/`, `routes/`, `database/`. Quelques centaines de kilooctets. En cas de nouvelle migration :

```
TERMINAL
cd ~/laravel_sandy && php artisan migrate --force
```

Modifier le site React

```
POWERSHELL
cd e:\Pjets_dan\Sandy\frontend
npm run build
```

Puis remplacer le contenu de `public_html/`.

Ajouter une dépendance PHP

Après un `composer require` en local, envoyer `composer.json` et `composer.lock`, puis :

```
TERMINAL
cd ~/laravel_sandy && composer install --no-dev --optimize-autoloader
```

> **NE JAMAIS LANCER CONFIG:CACHE**
>
> Cette commande met la configuration en cache, et à partir de là **tout appel à `env()` en dehors des fichiers `config/` renvoie `null`**. Le site tombe en panne sans message exploitable. Sans ce cache, Laravel est à peine plus lent et le piège est évité.
>
> Si c'est déjà fait : `php artisan config:clear`

Dépannage

SYMPTÔME	CAUSE LA PLUS PROBABLE	OÙ REGARDER
Erreur 500 sur `/api/ping`	Permissions manquantes sur `storage`, ou `.env` absent	`laravel_sandy/storage/logs/laravel.log`
Page 403 ou index des fichiers	Document Root ne pointe pas sur `/public`	cPanel → Domains → étape 2.5
Site vide, erreurs `localhost:8000` en console	Build fait avant la création de `.env.production`	Reprendre l'étape 1.4
Erreur CORS dans la console	`FRONTEND_URL` ne correspond pas exactement à l'adresse du site	`.env`, étape 4.3 — attention au `/` final
`could not find driver`	`pdo_mysql` absent du PHP en ligne de commande	Étape 4.1 — utiliser le chemin complet vers php 8.2
Les photos ne s'affichent pas	`APP_URL` incorrect, ou droits sur `public/images/products`	Étapes 4.3 et 4.6
Téléversement de photo en échec	Extension `fileinfo` désactivée	Étape 2.2
Connexion admin impossible	Mot de passe chiffré deux fois à la création	Recréer le compte, étape 4.5

> **LE RÉFLEXE QUI FAIT GAGNER LE PLUS DE TEMPS**
>
> Un code HTTP — 401, 419, 422, 500 — signifie que le serveur a répondu : le problème est dans le code ou dans la requête. **Aucun code HTTP**, en revanche, signifie que rien n'écoute en face : mauvais domaine, DNS non propagé, ou certificat absent.

Ce qui reste après la mise en ligne

- **Les photos produits** — à charger une par une depuis le back-office
- **Les pages légales** — mentions légales, conditions de livraison et de retour, confidentialité. Exigées par BANGE pour l'ouverture d'un TPV virtual
- **Un back-office des commandes** — aujourd'hui, consulter une commande ou la marquer comme payée passe encore par phpMyAdmin

Ce dernier point est le plus urgent une fois le site en ligne : tant qu'il manque, mieux vaut ne communiquer l'adresse à personne.
